

Reviewer Pack — Schur Coefficient Density Inverse Proportional to Frege Proo...

Entry 2b3f03ad1b52 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 03:10:24 UTC

Schur Coefficient Density Inverse Proportional to Frege Proof Length

Entry ID: 2b3f03ad1b52 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 03:10:24 UTC

1. Conjecture

Field A (mathematical branch): Symmetric Function Theory **Field B** (complexity object): Frege Proof Length

Statement:

For a homogeneous polynomial f of degree d in n variables, let $S(f)$ denote the number of non-zero coefficients in its Schur decomposition. Then, the Frege proof length of $f = 0$ is at least $\Omega(S(f)^{1/2})$

Rationale (proposer's reasoning):

The density of Schur coefficients reflects the complexity of the polynomial's representation, which might correlate with the difficulty of proving its identity through Frege systems. Symmetric function theory provides a framework to decompose polynomials into irreducible components, exposing structural properties that could influence proof complexity.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0d3b68b07bcd5ca9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random CNF instance with n variables and m clauses
    n = 30
    m = 10 * n
    cnf_instance = []
    for _ in range(m):
        clause = [random.randint(1, n), -random.randint(1, n)]
        cnf_instance.append(clause)

    # Map the CNF instance to a symmetric polynomial
    variables = set()
    for clause in cnf_instance:
        for var in clause:
            if var > 0:
                variables.add(var)
    n_vars = len(variables)

    def monomial_to_coeff(monomial):
        coeff = 1
        for var, exp in monomial.items():
            coeff *= var ** exp
        return coeff

    polynomial = {}
    for clause in cnf_instance:
        monomial = {var: 1 for var in clause}
        coeff = monomial_to_coeff(monomial)
        if coeff in polynomial:
            polynomial[coeff] += 1
        else:
            polynomial[coeff] = 1

    # Compute Schur decomposition coefficients using Young tableaux (simplified version)
```

```

def schur_decomposition(poly):
    schur_coeffs = {}
    for coeff, count in poly.items():
        schur_coeffs[count] = schur_coeffs.get(count, 0) + count
    return schur_coeffs

schur_coeffs = schur_decomposition(polynomial)

# Count non-zero entries (S(f))
S_f = sum(schur_coeffs.values())

# Measure Frege proof length via a DPLL-based proof generator (simplified version)
def dpll_proof_length(cnf):
    stack = []
    assignment = [False] * n_vars
    for clause in cnf:
        if all(not assignment[abs(var) - 1] == (var > 0) for var in clause):
            return 1
        for var in clause:
            if not assignment[abs(var) - 1]:
                stack.append((var, assignment[:]))
                assignment[abs(var) - 1] = var > 0
                break
    while stack:
        var, prev_assignment = stack.pop()
        assignment[abs(var) - 1] = not assignment[abs(var) - 1]
        if all(not prev_assignment[abs(var) - 1] == (var > 0) for var in clause):
            return len(stack) + 2
    return float('inf')

proof_length = dpll_proof_length(cnf_instance)

# Verify if proof length ≥ c * sqrt(S(f)) for some constant c > 0
if proof_length == float('inf'):
    conjecture_holds = False
    counterexample = "DPLL proof failed"
else:
    c = 1.0
    conjecture_holds = proof_length >= c * math.sqrt(S_f)
    counterexample = ""

return {
    "metric_name": "Frege Proof Length",
    "metric_value": proof_length,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2*i + 3 for i in range(5, 8)] # Default to first

    results = []
    for seed in seeds:
        result = run_trial(seed)

```

```

print(f"TRIAL: {result}")
results.append(result)

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif sum(1 for r in results if not r["conjecture_holds"]) / len(results) < 0.2:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"DPLL proof failed\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

ue': 1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'Frege Proof Length', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'Frege Proof Length', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'Frege Proof Length', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'Frege Proof Length', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'Frege Proof Length', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'Frege Proof Length', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'Frege Proof Length', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'Frege Proof Length', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'Frege Proof Length', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'Frege Proof Length', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': ''}

```

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_98998d42.py", line 121, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_98998d42.py", line 121, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~^

```

KeyError: 'seed'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to KeyError before producing reliable data | next: Fix the test code to handle missing 'seed' field and re-run with proper error handling

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111572
2	propose	ollama_remote	qwen3:8b	0	0	112669
3	propose	ollama_remote	qwen3:8b	0	0	144638
4	propose	ollama_remote	qwen3:8b	0	0	95109
5	preregistration	ollama_remote	qwen3:8b	0	0	24112
6	novelty	ollama_remote	qwen3:8b	0	0	21967
7	novelty	ollama_remote	qwen3:8b	0	0	14100
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19599
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11600
10	judge	ollama_remote	qwen3:8b	0	0	17211

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 572576 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/2b3f03ad1b52.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/2b3f03ad1b52.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/2b3f03ad1b52.tar.gz (if generated)